Pruebas unitarias

Nombre del proyecto	ADOPTAFACIL
Nombre del sistema	Sistema de información para la adopción de animales
Versión	1.0
Equipo	Nicolas Alberto Valenzuela Pacheco Brayan Esteban Sanchez Avila Andres Felipe Gamez Arias Midred Callejas Chaparro
Instructor	Ruben Sanabria

1. Introducción
PARTE I. Fundamentos teóricos de las pruebas unitarias
2. Definición de pruebas unitarias
3. Objetivos de las pruebas unitarias
    3.1. Objetivo general
    3.2. Objetivos específicos
4. Herramientas utilizadas
    4.1. PestPHP
PARTE II. Implementación de pruebas unitarias en AdoptaFácil
5. Arquitectura de pruebas en el proyecto
6. Pruebas sobre políticas de autorización
    6.1. Descripción
7. Casos de prueba: PostPolicy
    7.1. Escenarios evaluados
8. Ejemplo de código de prueba
9. Validación de permisos de usuarios
10. Pruebas de seguridad en carga de archivos
    10.1. Descripción
11. Casos de prueba: carga segura de archivos
12. Ejemplo de código de prueba

13. Validación de nombres aleatorios
14. Prueba de archivos maliciosos
15. Resultados obtenidos
16. Beneficios de las pruebas implementadas
17. Conclusiones

---

1. Introducción

El aseguramiento de la calidad del software es una fase fundamental dentro del ciclo de desarrollo de sistemas informáticos. Una de las prácticas más importantes dentro de este proceso es la implementación de **pruebas unitarias**, las cuales permiten validar el funcionamiento correcto de componentes individuales del software.

El presente documento describe el proceso de **diseño, implementación y ejecución de pruebas unitarias en el proyecto AdoptaFácil**, una aplicación web desarrollada utilizando el framework **Laravel**. Estas pruebas han sido implementadas utilizando **PestPHP**, una herramienta moderna de testing para aplicaciones PHP.

Las pruebas unitarias desarrolladas en el proyecto se enfocan principalmente en dos áreas críticas del sistema:

- **Políticas de autorización (Policies)** utilizadas para controlar permisos de acceso a recursos del sistema.
- **Seguridad en la carga de archivos**, implementada mediante el trait **SecureFileUpload**, cuyo objetivo es prevenir cargas de archivos maliciosos o no válidos.

PARTE I. Fundamentos teóricos de las pruebas unitarias

2. Definición de pruebas unitarias

Las **pruebas unitarias** son un tipo de prueba de software que tiene como objetivo verificar el correcto funcionamiento de las unidades más pequeñas de un programa.

Una unidad puede ser:

- Un método.

- Una función.
- Una clase.
- Un módulo específico.

El propósito principal de estas pruebas es asegurar que cada componente del sistema funcione correctamente de forma **independiente del resto del sistema**.

Según el enfoque de desarrollo moderno, las pruebas unitarias deben:

- Ejecutarse de forma automática.
- Ser rápidas.
- Ser repetibles.
- Validar comportamientos específicos del código.

En el caso de aplicaciones desarrolladas con **Laravel**, las pruebas unitarias permiten validar la lógica interna del sistema sin necesidad de interactuar con interfaces externas como bases de datos o APIs.

3. Objetivos de las pruebas unitarias

3.1. Objetivo general

Garantizar que los componentes internos del sistema AdoptaFácil funcionen correctamente mediante la validación automatizada de su lógica interna.

3.2. Objetivos específicos

- Verificar el correcto funcionamiento de las **políticas de autorización**.
- Validar la **seguridad del sistema de carga de archivos**.
- Detectar errores de lógica durante el desarrollo.
- Facilitar el mantenimiento del código.
- Mejorar la confiabilidad del sistema.

4. Herramientas utilizadas

Las pruebas unitarias del proyecto fueron desarrolladas utilizando la herramienta **PestPHP**.

4.1. PestPHP

PestPHP es un framework moderno de testing para PHP basado en PHPUnit que permite escribir pruebas de forma más sencilla y legible.

Características principales:

- Sintaxis simple y expresiva.
- Integración con Laravel.
- Ejecución rápida.
- Compatibilidad con PHPUnit.

Ejemplo de sintaxis en Pest:

```
test('example test', function () {
    expect(1 + 1)->toBe(2);
});
```

PARTE II. Implementación de pruebas unitarias en AdoptaFácil

5. Arquitectura de pruebas en el proyecto

Las pruebas del proyecto AdoptaFácil están organizadas dentro del directorio `tests/`.

Estructura relevante para pruebas unitarias:

```
tests
├── Unit
│   ├── PoliciesTest.php
│   └── SecureFileUploadTest.php
```

Cada archivo de prueba se encarga de validar un conjunto específico de funcionalidades del sistema.

6. Pruebas sobre políticas de autorización

6.1. Descripción

Las **Policies** en Laravel permiten definir reglas de autorización para determinar qué usuarios pueden realizar determinadas acciones sobre un recurso.

En el proyecto AdoptaFácil se definieron pruebas unitarias para las siguientes políticas:

- **PostPolicy**.
- **ProductPolicy**.
- **MascotaPolicy**.

Estas pruebas verifican que los permisos de acceso se comporten correctamente según el tipo de usuario.

7. Casos de prueba: PostPolicy

La prueba verifica diferentes acciones que los usuarios pueden realizar sobre publicaciones.

7.1. Escenarios evaluados

Caso	Descripción
Ver publicaciones activas	Permitido
Ver publicaciones inactivas	No permitido
Crear publicación	Permitido
Actualizar publicación propia	Permitido
Actualizar publicación de otro usuario	Denegado
Eliminar publicación propia	Permitido
Eliminar publicación como administrador	Permitido

8. Ejemplo de código de prueba

Fragmento del archivo **PoliciesTest.php**:

```
test('post policy rules are enforced', function () {
    $policy = new PostPolicy();
    $activePost = new Post(['is_active' => true, 'user_id' =>
10]);
    $inactivePost = new Post(['is_active' => false, 'user_id'
=> 10]);

    expect($policy->viewAny(null))->toBeTrue();
    expect($policy->view(null, $activePost))->toBeTrue();
    expect($policy->view(null, $inactivePost))->toBeFalse();
});
```

Este fragmento verifica que:

- Las publicaciones activas pueden visualizarse.
- Las publicaciones inactivas no pueden visualizarse.

9. Validación de permisos de usuarios

La prueba también evalúa diferentes tipos de usuarios.

Ejemplo de creación de usuarios de prueba:

```
$author = new User();
$author->id = 10;
$author->role = 'cliente';

$otherUser = new User();
$otherUser->id = 11;
$otherUser->role = 'cliente';

$admin = new User();
$admin->id = 12;
$admin->role = 'admin';
```

Esto permite verificar que:

- El **autor del contenido** puede modificarlo.
- Otros usuarios no pueden modificar contenido ajeno.
- Los **administradores** poseen permisos especiales.

10. Pruebas de seguridad en carga de archivos

10.1. Descripción

El sistema implementa un trait llamado **SecureFileUpload** cuyo propósito es garantizar que los archivos subidos al sistema cumplan con condiciones de seguridad.

Las pruebas unitarias verifican:

- Que las imágenes válidas se carguen correctamente.
- Que los archivos con contenido inválido sean rechazados.

11. Casos de prueba: carga segura de archivos

Caso	Descripción
Subir imagen válida	Permitido
Generación de nombre aleatorio	Correcto
Validación de extensión	Correcta
Rechazo de archivos inválidos	Correcto

12. Ejemplo de código de prueba

Fragmento del archivo **SecureFileUploadTest.php**:

```
test('secure upload stores a valid image with randomized file
name', function () {
    Storage::fake('public');
    $uploader = new TestSecureUploader();
    $file = UploadedFile::fake()->create('avatar.png', 10);

    $storedPath = $uploader->upload($file, 'avatars', 'publi
```

```
c');

    expect($storedPath)->toBeString();
});
```

Esta prueba verifica que:

- El archivo se almacene correctamente.
- El sistema genere una ruta válida.

13. Validación de nombres aleatorios

El sistema genera nombres de archivo aleatorios para evitar:

- Colisiones de nombres.
- Acceso predecible a archivos.

Validación realizada en la prueba:

```
expect(pathinfo($storedPath, PATHINFO_FILENAME))
    ->toMatch('/^[0-9a-f\-]{36}$/i');
```

Esto confirma que el nombre del archivo corresponde a un **UUID aleatorio**.

14. Prueba de archivos maliciosos

Otra prueba verifica que archivos con **contenido inválido** sean rechazados.

Ejemplo:

```
$file = UploadedFile::fake()->create(
    'not-really-an-image.jpg',
    10,
    'image/jpeg'
);
```

Aunque el archivo aparenta ser una imagen, su contenido no lo es, por lo que el sistema debe rechazarlo.

Resultado esperado:

```
expect($storedPath)->toBeFalse();
```

15. Resultados obtenidos

Después de ejecutar las pruebas unitarias del sistema se obtuvieron los siguientes resultados.

Tipo de prueba	Resultado
Policies	Correcto
Validación de permisos	Correcto
Carga segura de archivos	Correcto
Validación de contenido	Correcto
Generación de nombres aleatorios	Correcto

No se detectaron errores críticos durante la ejecución de las pruebas unitarias.

16. Beneficios de las pruebas implementadas

La implementación de pruebas unitarias en el proyecto proporciona múltiples beneficios:

- Mayor confiabilidad del sistema.
- Prevención de errores en producción.
- Validación automática del código.
- Mayor facilidad para mantenimiento del sistema.
- Mejora en la seguridad de la aplicación.

17. Conclusiones

Las pruebas unitarias implementadas en el proyecto AdoptaFácil permiten garantizar la correcta funcionalidad de componentes críticos del sistema, especialmente aquellos relacionados con **autorización y seguridad de archivos**.

El uso de **PestPHP** ha permitido desarrollar pruebas claras, legibles y fáciles de mantener, lo que contribuye significativamente al aseguramiento de la calidad del software.

Los resultados obtenidos demuestran que:

- Las políticas de autorización funcionan correctamente.
- El sistema de carga de archivos previene la subida de archivos maliciosos.
- El comportamiento esperado del sistema se mantiene estable.

La implementación de estas pruebas constituye una práctica fundamental dentro del desarrollo del proyecto y establece una base sólida para futuras mejoras y ampliaciones del sistema.